



Threads

Prof. Pedro Botelho

Universidade Federal do Ceará

15 de abril de 2026

Conteúdo



Introdução a Multithreading

Modelagem da Thread

Utilizando Threads

Implementação de Threads

Conclusão



Introdução a Multithreading

Modelagem da Thread

Utilizando Threads

Implementação de Threads

Conclusão



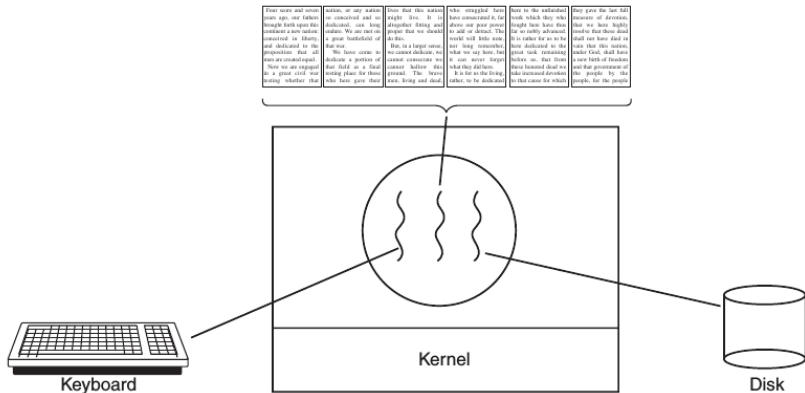
- ▶ SOs tradicionais: única thread de controle
- ▶ SOs modernos: múltiplas threads
- ▶ Também denominadas miniprocessos
- ▶ **Por que alguém iria querer ter um tipo de processo dentro de um processo?**
 - ▶ Múltiplas atividades dentro de uma mesma aplicação
 - ▶ Simplificação do modelo de programação
 - ▶ Sem chaveamento de contexto
- ▶ Atividades paralelas compartilhando endereçamento
- ▶ Criação/Destruição mais simples (e rápida)
- ▶ Desempenho
 - ▶ CPU bound → não há ganho
 - ▶ CPU + E/S equivalentes → aceleração da aplicação

Exemplo: Editor de texto



- ▶ Arquivo de 800 páginas
- ▶ Ações do usuário
 - ▶ Remoção de conteúdo na página 1
 - ▶ Em seguida, visualizar página 600
- ▶ Ações requeridas para a aplicação
 - ▶ Reformatar todo documento até a página 600
 - ▶ Apresentar página 600 após formatação
- ▶ Atraso na apresentação da página 600
- ▶ **SOLUÇÃO:** Modelo com duas threads:
 - ▶ Interação com o usuário
 - ▶ Formatação em segundo plano
- ▶ Possível terceira thread para salvar o arquivo

Exemplo: Editor de texto

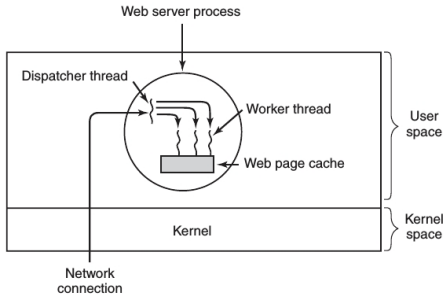


Exemplo: Servidor Web



- ▶ Despachante
 - ▶ Laço infinito recebendo requisições
 - ▶ Ativa operário
- ▶ Operário
 - ▶ Laço infinito recebendo requisições
 - ▶ Se página no cache, entrega ao despachante
 - ▶ Caso contrário, acessa disco

Exemplo: Servidor Web



```
while (TRUE) {  
    get_next_request(&buf);  
    handoff_work(&buf);  
}
```

```
while (TRUE) {  
    wait_for_work(&buf)  
    look_for_page_in_cache(&buf, &page);  
    if (page_not_in_cache(&page))  
        read_page_from_disk(&buf, &page);  
    return_page(&page);  
}
```




Introdução a Multithreading

Modelagem da Thread

Utilizando Threads

Implementação de Threads

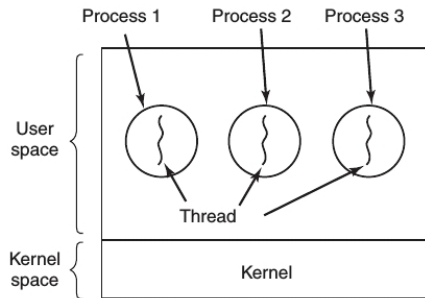
Conclusão

Modelo de Thread Clássico

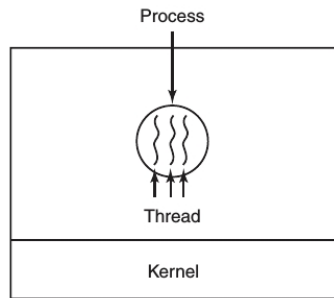


- ▶ Thread dependente do processo
- ▶ Conceitos distintos
 - ▶ Processo: agrupamento de recursos
 - ▶ Thread: contador de programa que controla qual instrução deve ser executada em seguida
- ▶ Múltiplas execuções dentro do processo
- ▶ Múltiplas Threads em um processo $\leftrightarrow$ Múltiplas Processos em um computador
- ▶ Threads compartilham endereçamento $\leftrightarrow$ Processos compartilham memória físicas, discos, impressoras e outros recursos
- ▶ Threads Também chamadas de processos leves (*lightweight process*)
- ▶ CPUs *multithread* $\rightarrow$ Suporte para chaveamento de *multithreads*

Modelo de Thread Clássico



(a)



(b)

Modelo de Thread Clássico



- ▶ Dependência entre Threads
 - ▶ Não há proteção
 - ▶ Cooperação
- ▶ Mesma máquina de estados dos processos
- ▶ Criação e destruição similares aos processos
- ▶ Não existe hierarquia entre threads
- ▶ Problemas:
 - ▶ Heranças entre processos
 - ▶ Compartilhamento de dados
- ▶ Cuidados específicos no projeto

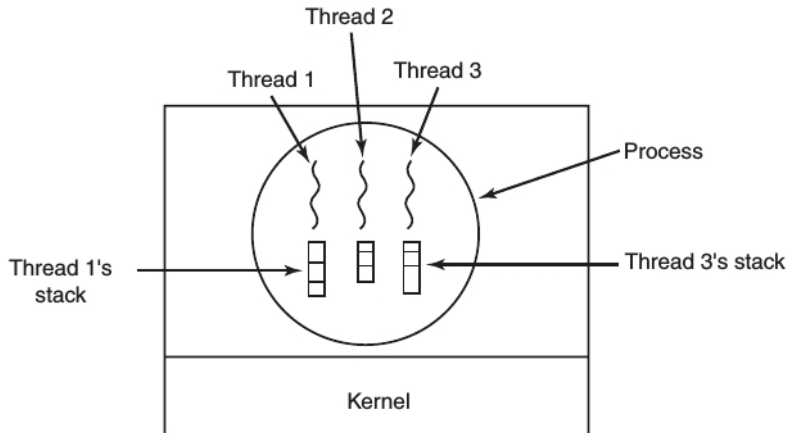
Modelo de Thread Clássico



A primeira coluna lista alguns itens compartilhados por todos os threads em um processo. A segunda lista alguns itens específicos a cada thread.

Itens por processo	Itens por thread
Espaço de endereçamento	Contador de programa
Variáveis globais	Registradores
Arquivos abertos	Pilha
Processos filhos	Estado
Alarmes pendentes	
Sinais e tratadores de sinais	
Informação de contabilidade	

Modelo de Thread Clássico



Conteúdo



Introdução a Multithreading

Modelagem da Thread

Utilizando Threads

Implementação de Threads

Conclusão



- ▶ Padrão IEEE 1003.1c
- ▶ Portabilidade
- ▶ Pacote *Pthreads* (mais de 60 funções)
- ▶ Propriedades de uma thread *Pthreads*:
 - ▶ Identificador
 - ▶ Registros
 - ▶ Atributos

Funções Importantes



Chamada de thread	Descrição
Pthread_create	Cria um novo thread
Pthread_exit	Conclui a chamada de thread
Pthread_join	Espera que um thread específico seja abandonado
Pthread_yield	Libera a CPU para que outro thread seja executado
Pthread_attr_init	Cria e inicializa uma estrutura de atributos do thread
Pthread_attr_destroy	Remove uma estrutura de atributos do thread

Exemplo de Código



```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>

#define NUMBER_OF_THREADS 10

void *print_hello_world(void *tid)
{
    /* Esta função imprime o identificador do thread e sai. */
    printf("Hello World. Greetings from thread %d\n", tid);
    pthread_exit(NULL);
}

int main(int argc, char *argv[])
{
    /* O programa principal cria 10 threads e sai. */
    pthread_t threads[NUMBER_OF_THREADS];
    int status, i;

    for(i=0; i < NUMBER_OF_THREADS; i++) {
        printf("Main here. Creating thread %d\n", i);
        status = pthread_create(&threads[i], NULL, print_hello_world, (void *)i);

        if (status != 0) {
            printf("Oops. pthread_create returned error code %d\n", status);
            exit(-1);
        }
    }
    exit(NULL);
}
```



Introdução a Multithreading

Modelagem da Thread

Utilizando Threads

Implementação de Threads

Conclusão

Implementação de Threads



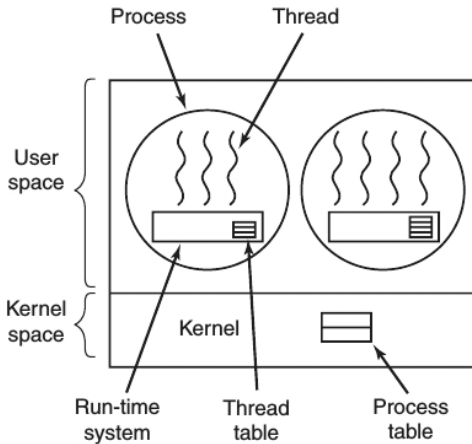
- ▶ Modo usuário
- ▶ Modo núcleo (kernel)
- ▶ Modo híbrido

Threads em Modo Usuário



- ▶ O primeiro método é colocar o **pacote de threads** inteiramente no **espaço do usuário** ⇒ **Núcleo** não é **informado** sobre sua **existência**
- ▶ Vantagem – o pacote de thread pode ser implementado em um **SO** que **não suporta threads**
- ▶ Threads são implementação através de uma **biblioteca**
- ▶ Quando os threads são gerenciados no espaço do usuário → Tabela de Threads
- ▶ Chaveamento com poucas instruções
- ▶ Escalonamento personalizado
- ▶ Problemas:
 - ▶ Sistemas bloqueantes
 - ▶ Thread deve ceder CPU voluntariamente

Threads em Modo Usuário

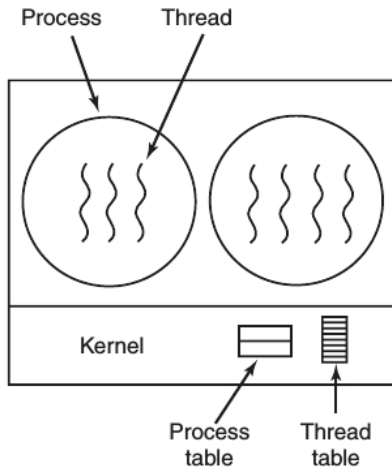


Threads em Modo Núcleo



- ▶ Núcleo gerencia as threads
- ▶ Tabela de threads do sistema
- ▶ Chamadas ao núcleo para criação/destruição
- ▶ Thread bloqueado
 - ▶ Mesmo processo
 - ▶ Mudar processo
- ▶ Custo maior → reciclagem

Threads em Modo Núcleo

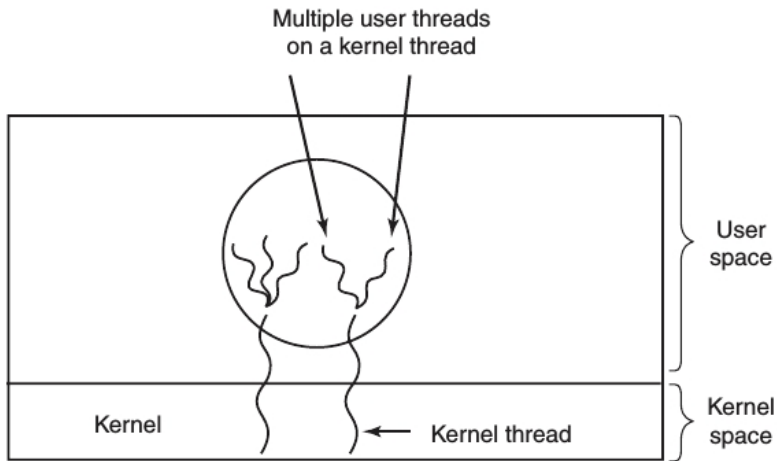


Threads em Modo Híbrido



- ▶ Combina duas abordagens
- ▶ Threads de núcleo $\leftrightarrow$ Threads de usuário
 - ▶ Multiplexação

Threads em Modo Híbrido



Conteúdo



Introdução a Multithreading

Modelagem da Thread

Utilizando Threads

Implementação de Threads

Conclusão

Dúvidas?



Obrigado pela atenção!